AWS EBS (Elastic Block Store).

1. What is AWS EBS?

AWS EBS (Elastic Block Store) is a block-level storage service provided by Amazon Web Services that is used with Amazon EC2 instances.

Think of it like a virtual hard disk attached to a cloud server.

- It stores persistent data.
- Even if the EC2 instance stops or crashes, the data remains safe.
- Mainly used for databases, file systems, and applications that require high performance storage.

2. Key Characteristics

1. Block Storage
   Data is stored in blocks (similar to SSD/HDD).
2. Persistent Storage
   Data remains even after instance restart.
3. Attach/Detach Feature
   EBS volumes can be attached or detached from EC2 instances.
4. Snapshot Support
   Backup of EBS volumes can be stored in Amazon S3 as snapshots.
5. High Availability
   Replicated automatically inside the same Availability Zone.

3. Types of EBS Volumes

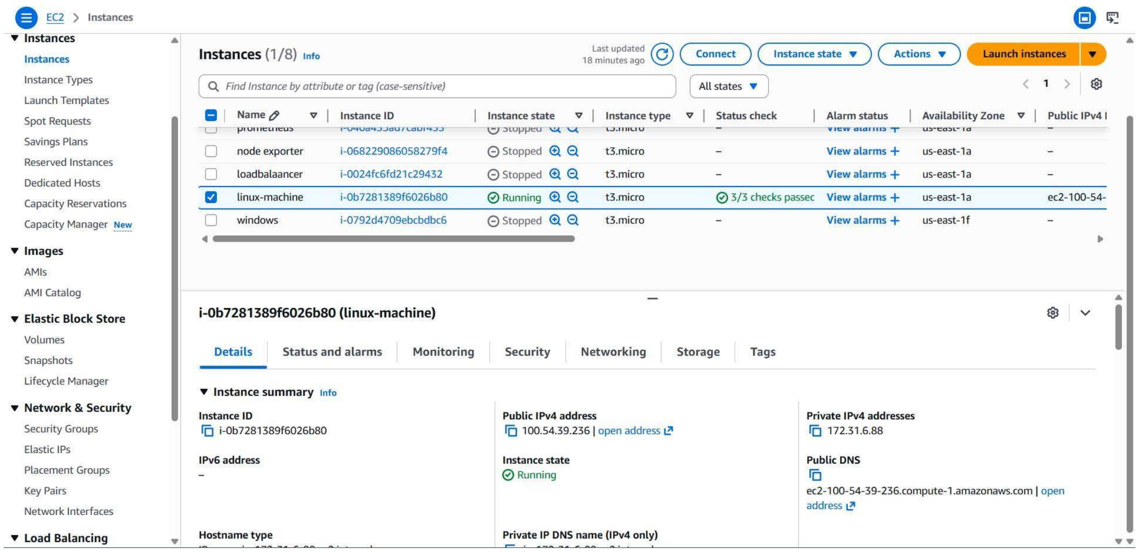
Volume Type Description Use Case

gp3 / gp2 General Purpose SSD Web servers, small databases io1 / io2 Provisioned IOPS SSD High-performance databases st1 Throughput Optimized HDD Big data, analytics

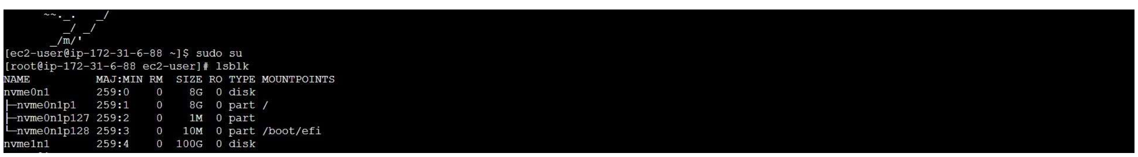
Volume Type Description Use Case

sc1	Cold HDD	Infrequent access

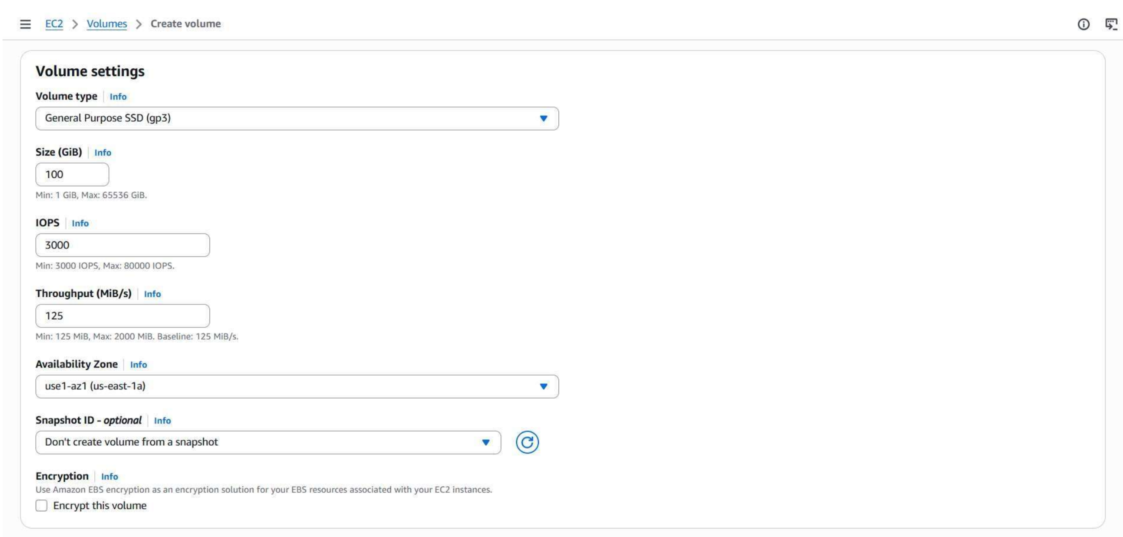
I have created an instance and then I have created an volume of 100 GB and attached to this instance.



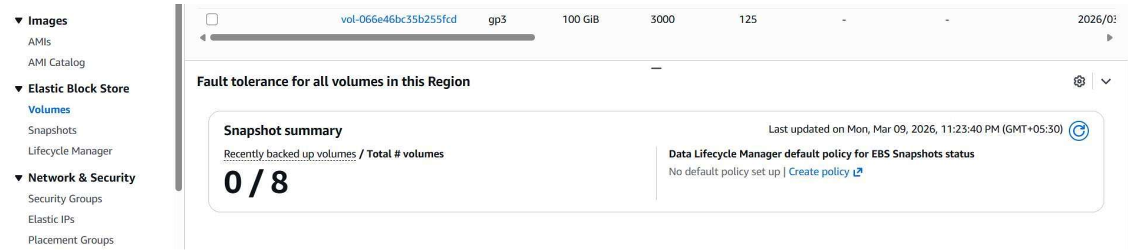
You can check using the lsblk command.



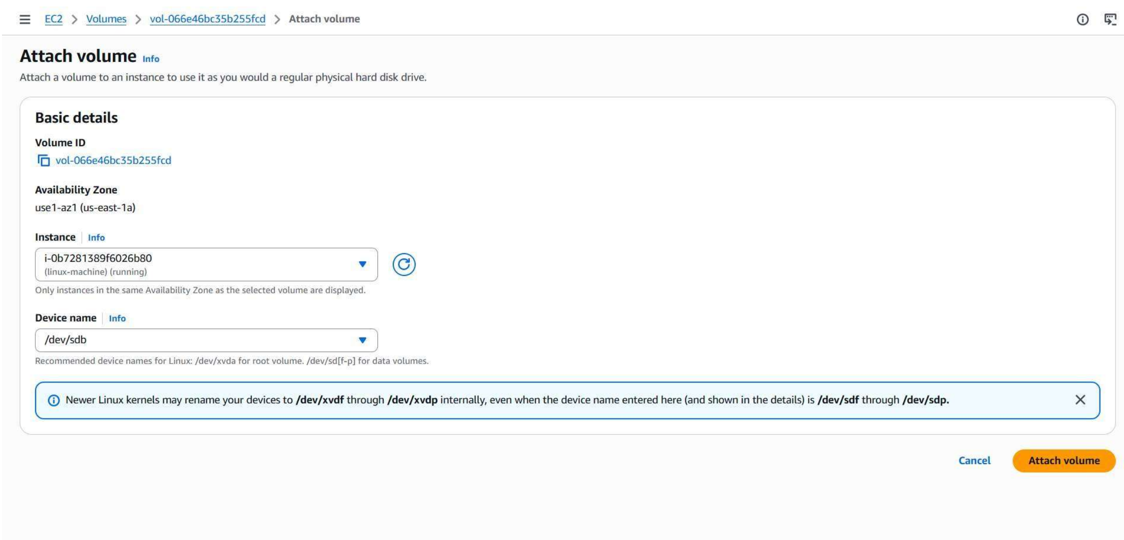
This is the created configuration storage of the new volume.



Created volume.

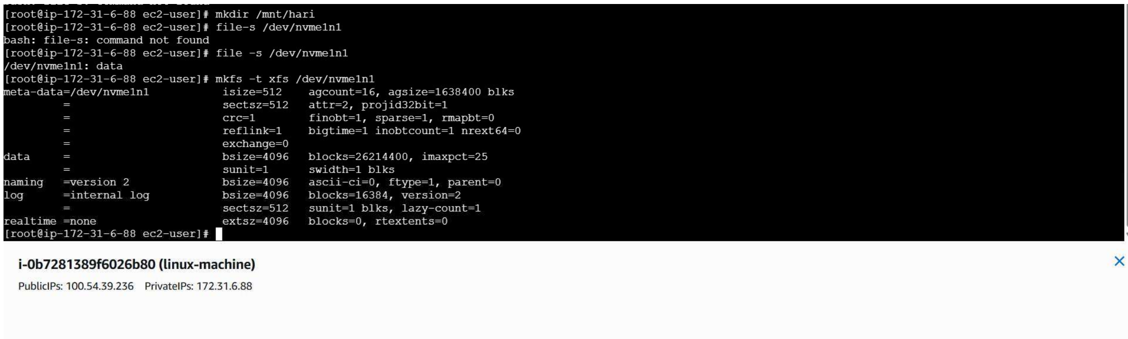


Then we have to attach to the instance but the EBS and EC2 have to be created in same region and same AZ's. Then we have to select the running ec2 machine.



When we use lsblk we can see the un mounted volume with free space.

file -s /dev/nvme1n1 >>>> Checks if the disk already has a filesystem. mkfs -t xfs /dev/nvme1n1 >>>> Creates a filesystem on the disk.

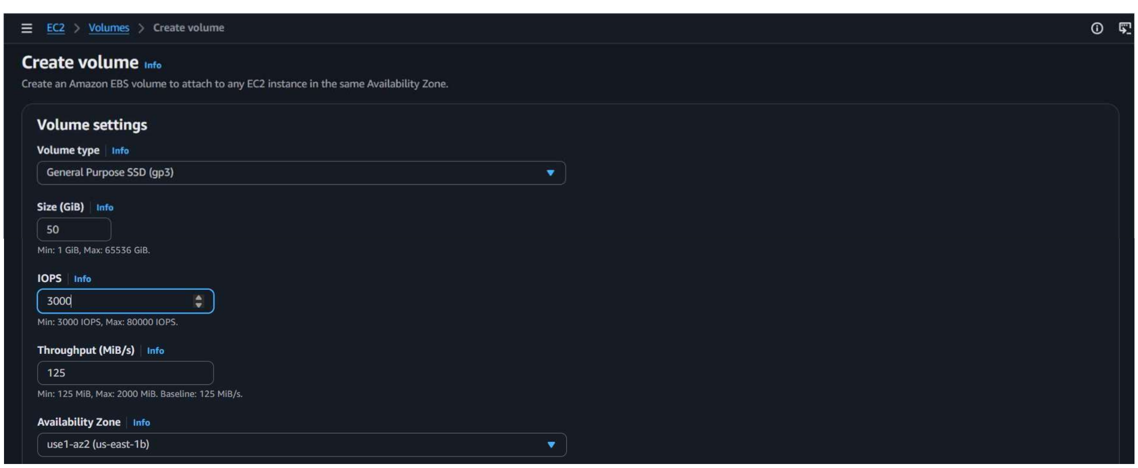


mkdir /mnt/myvolume Creates a mount point directory where the volume will be attached.

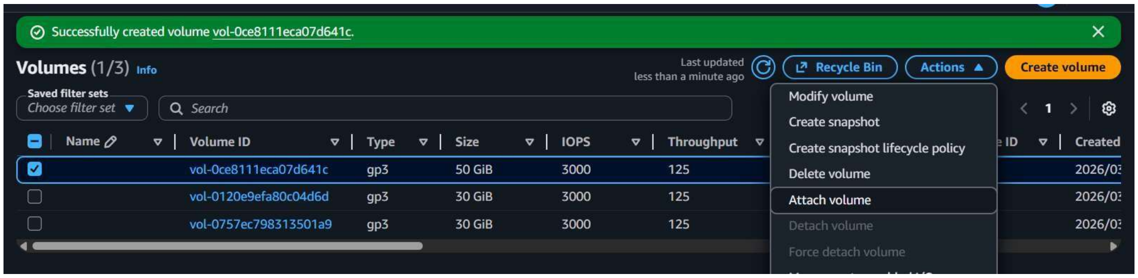
mount /dev/nvme1n1 /mnt/myvolume Mounts the EBS volume to the directory. df -h Shows disk usage and mounted filesystems.

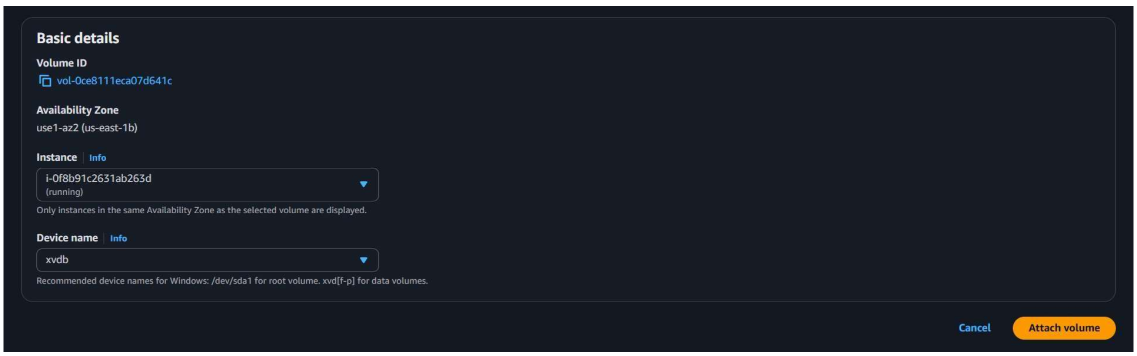
EBS in Windows instance.

First we have to create an EBS in the same region and AZ.

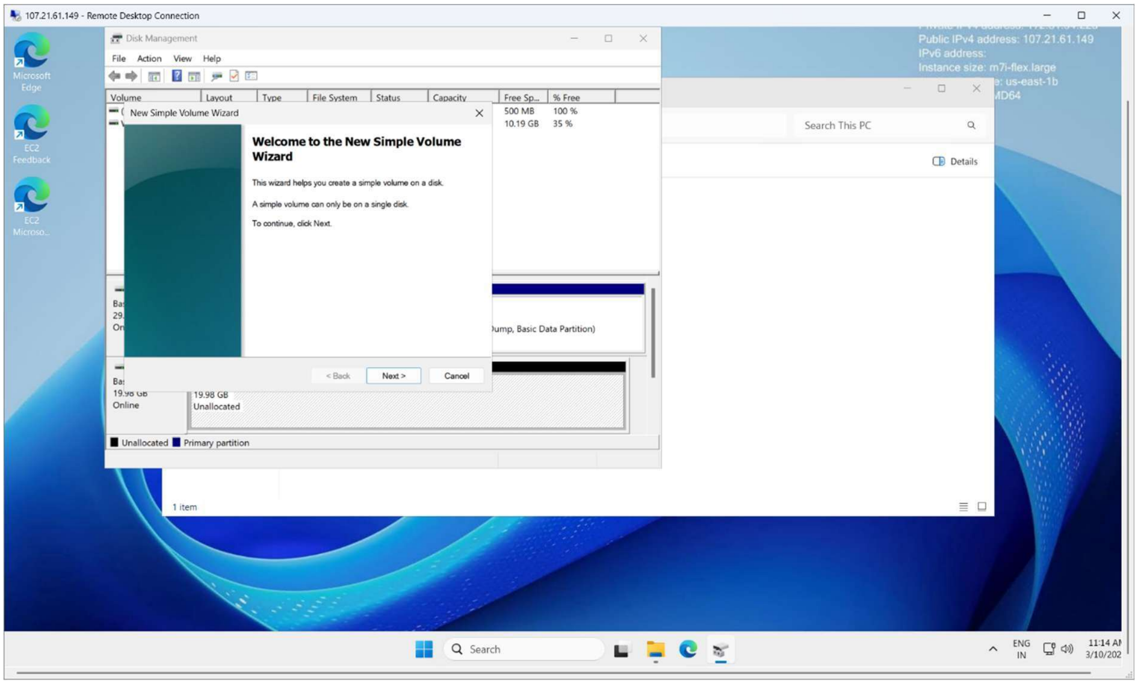


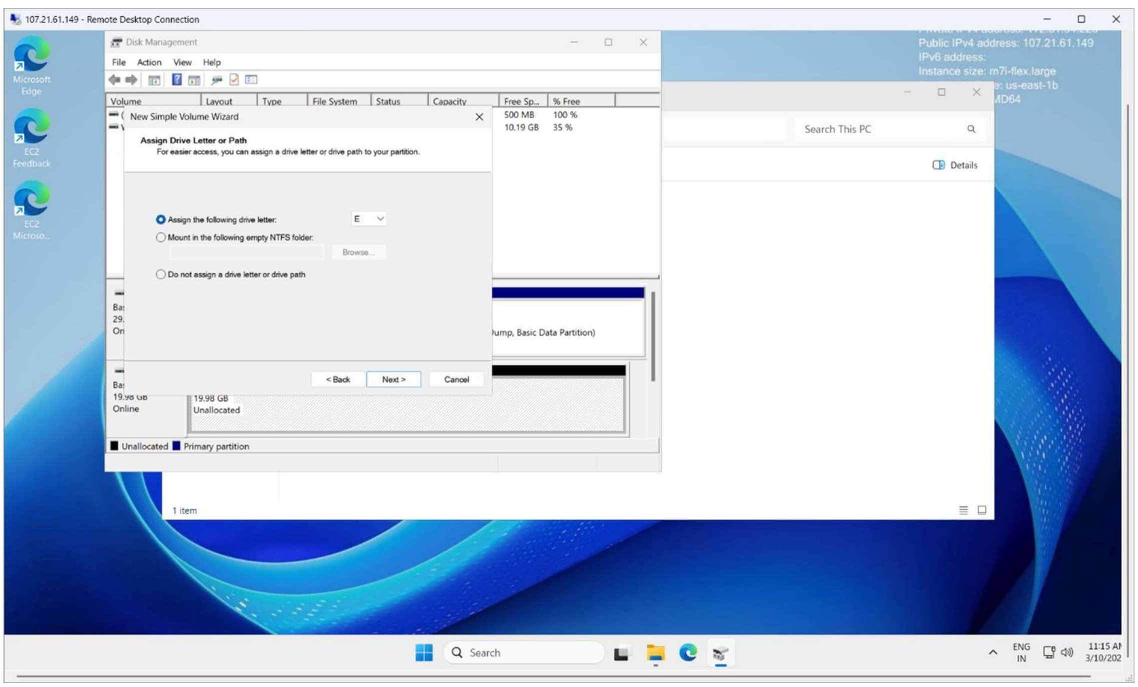
Then we have to attach the volume to the created instance.

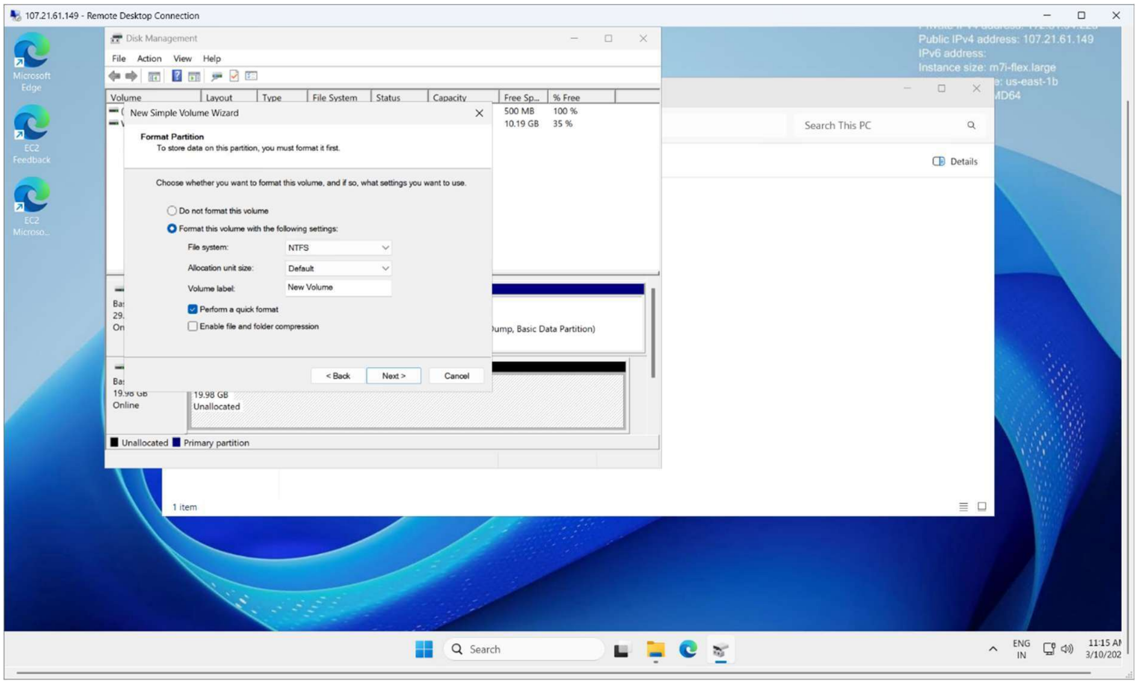




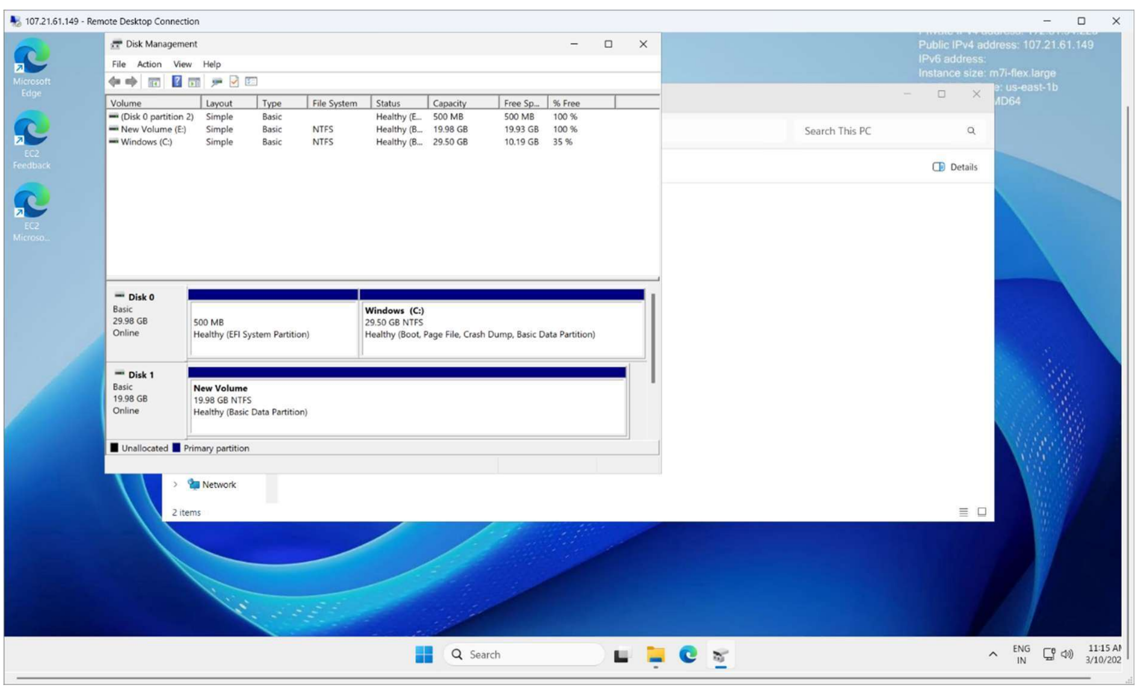
After searching for disk management you can see this app and you have to allocate the unallocated attached volume.


107.21.61.149 - Remote Desktop Connection
Disk Management
New Simple Volume Wizard
Welcome to the New Simple Volume Wizard
This wizard helps you create a simple volume on a disk.
A simple volume can only be on a single disk.
To continue, click Next.
< Back
Next >
Cancel
Public IPv4 address: 107.21.61.149
Instance size: m7i-flex.large
Unallocated
Primary partition

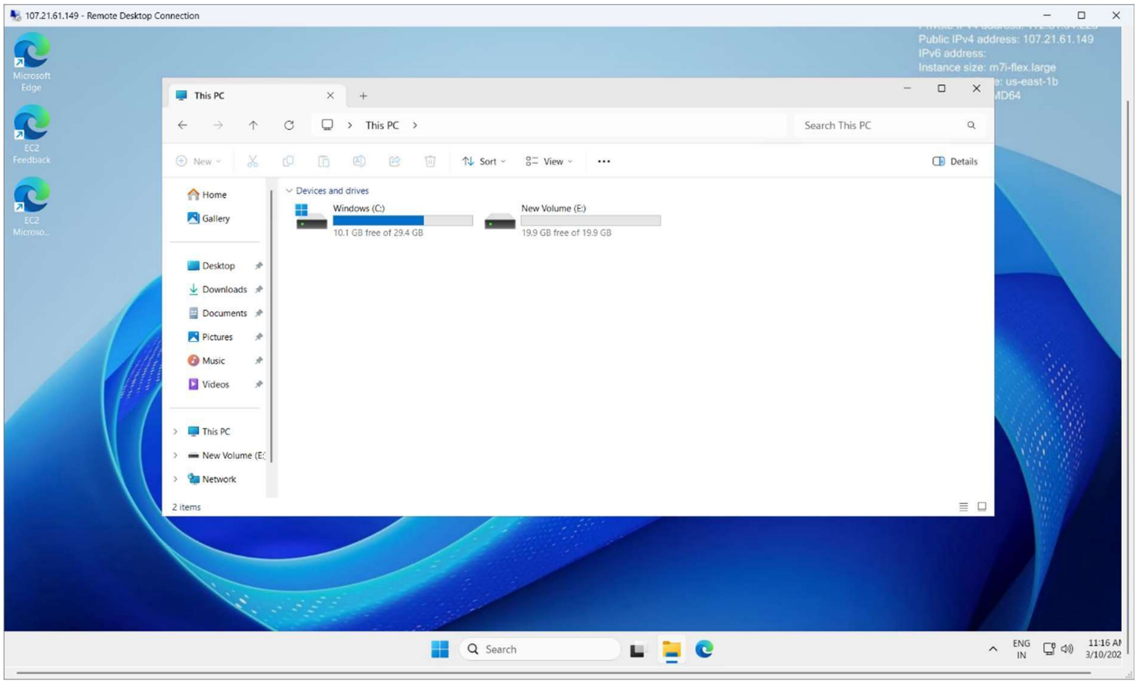

107.21.61.149 - Remote Desktop Connection
Disk Management
New Simple Volume Wizard
Assign Drive Letter or Path
For easier access, you can assign a drive letter or drive path to your partition.
Assign the following drive letter:
E
Mount in the following empty NTFS folder:
Browse...
Do not assign a drive letter or drive path
< Back
Next >
Cancel
Public IPv4 address: 107.21.61.149
Instance size: m7i-flex.large
Unallocated
Primary partition



Then I have done all the procedure and the volume has been formatted and allocated.



We can see the allocated disk.



<u>EBS Snapshot</u>

An EBS Snapshot is a backup of an EBS volume stored in Amazon S3.

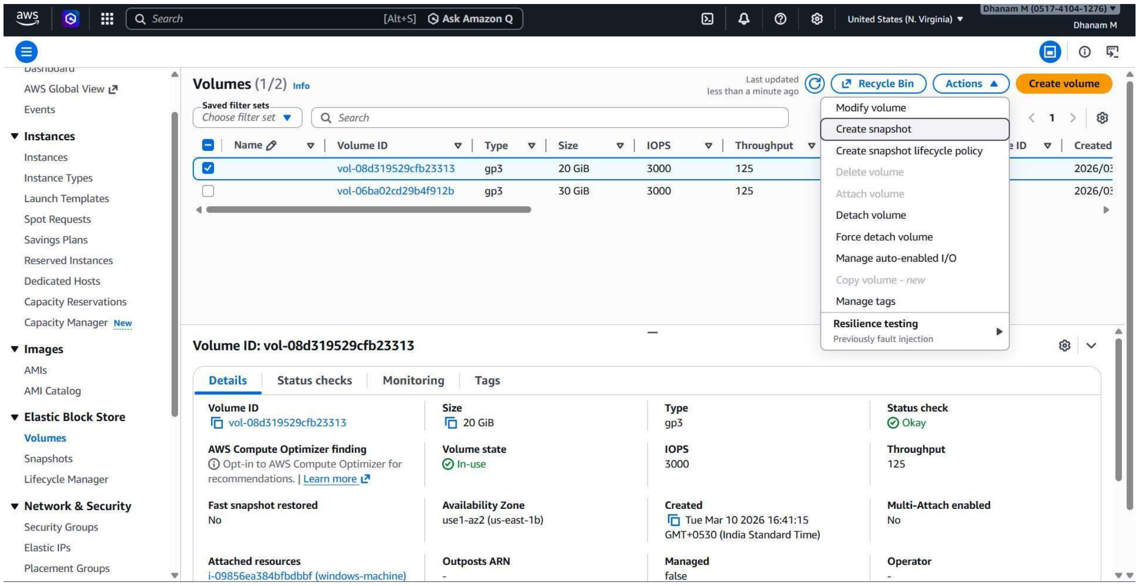
Key Points

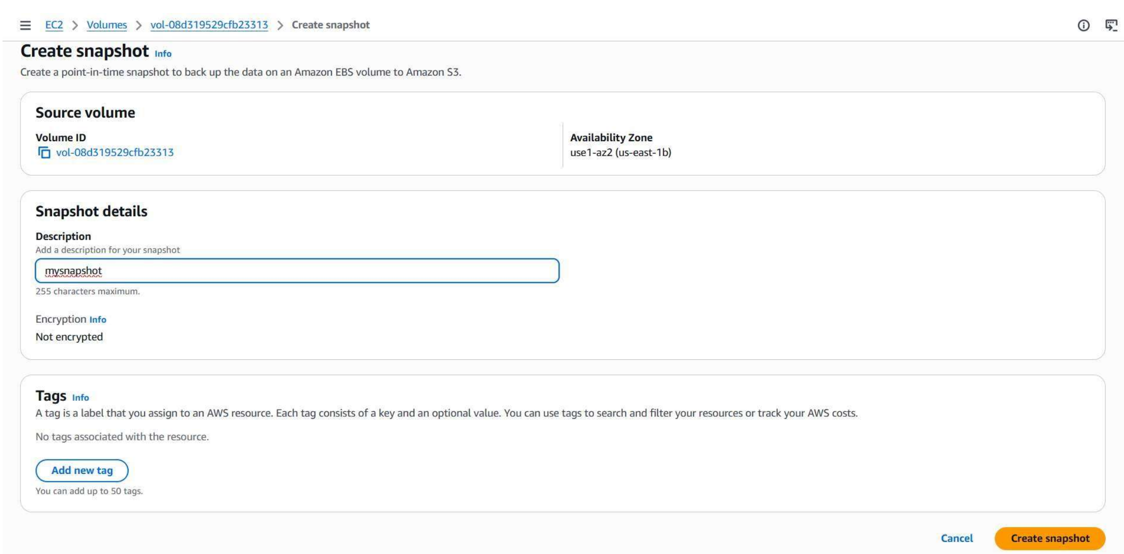
- It is used to backup data of an EBS volume.
- Snapshots are incremental (only changes are saved after the first snapshot).
- Used for data recovery.
- You can create a new EBS volume from a snapshot.

Example Flow

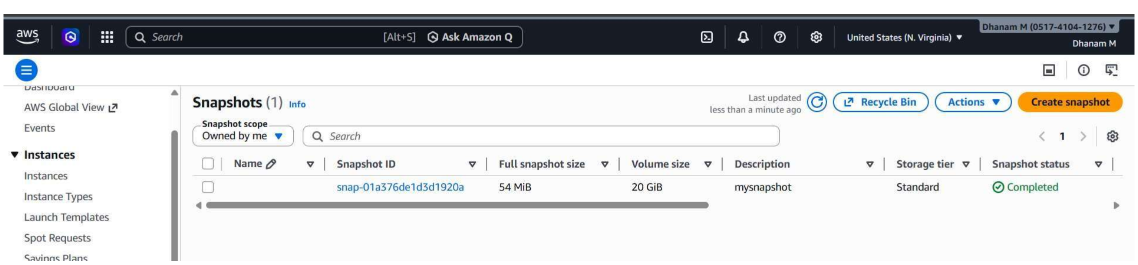
1. EC2 instance has an EBS volume.
2. You create a snapshot of that volume.
3. Later you can restore it → create a new EBS volume → attach it to an instance.

I have selected the volume and creating one snapshot.





Then I have the volume back (i.e) snapshot's



AMI (Amazon Machine Image)

An AMI is a complete template used to launch an EC2 instance.

It contains:

- OS (Linux / Windows)
- Application software
- Configuration
- Snapshots of attached volumes

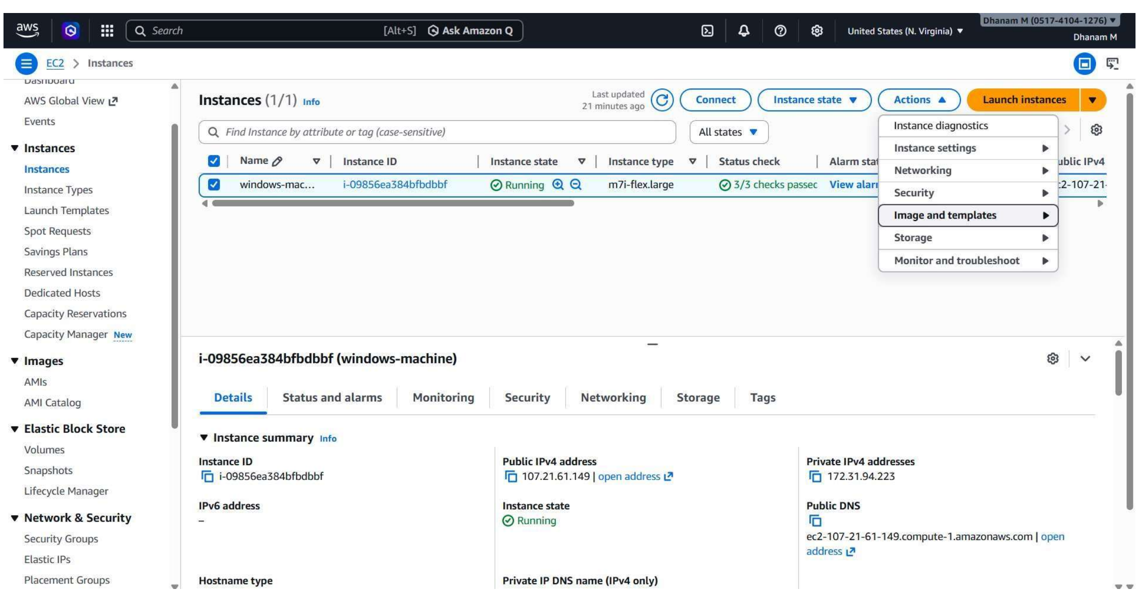
Key Points

- Used to launch new EC2 instances.
- Contains snapshots of EBS volumes.
- Acts like a golden template.

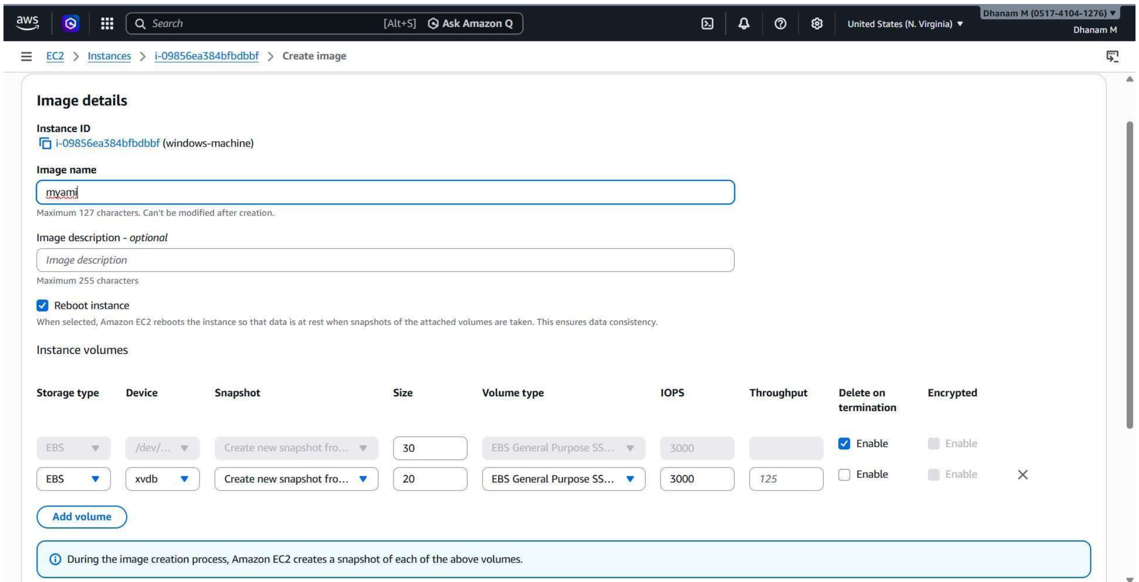
Example Flow

1. Configure an EC2 instance.
2. Install software (Apache, Docker, etc.).
3. Create an AMI from that instance.
4. Launch multiple instances using that AMI.

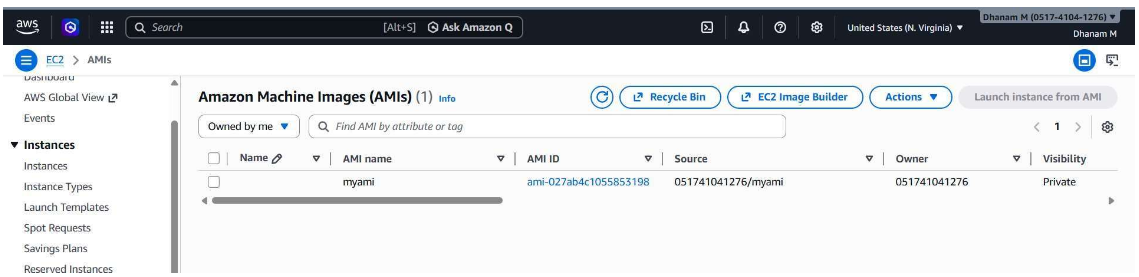
For creation of the image we can use the pre-installed webserver and any other services are build to AMI. By clicking the create image button for the selected instance



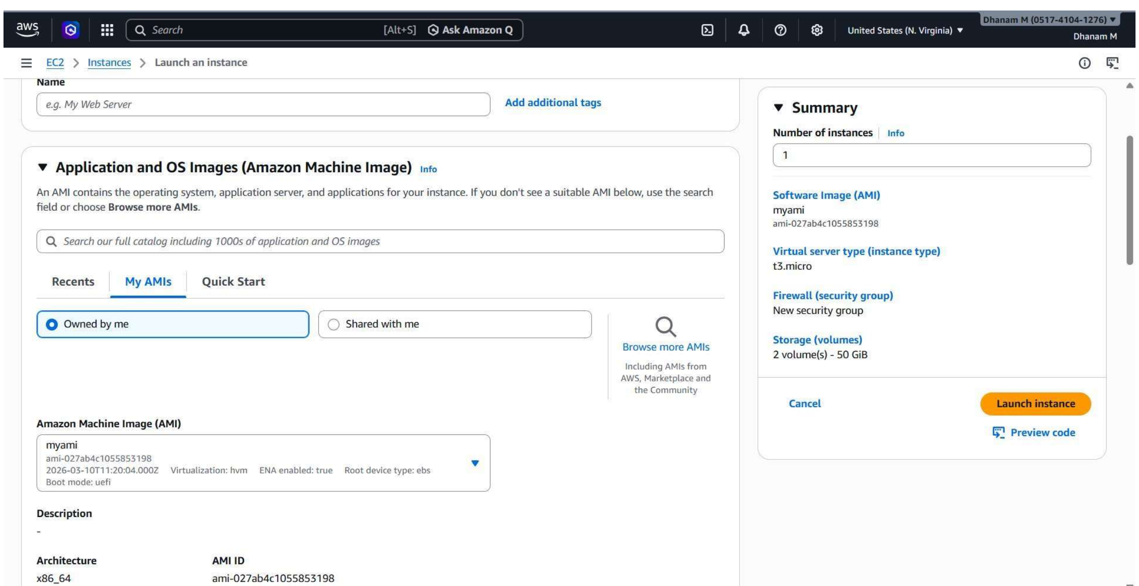
Then we have to name the MYAMI.



When the created MYAMI is available we can use it for launching an instance.



The created MYAMI is in the created by me section.



VPC Peering.

1. What VPC Peering Actually Is

VPC Peering is a network connection between two Virtual Private Clouds (VPCs) that allows resources in both VPCs to communicate privately using private IP addresses.

No internet gateway.
No NAT traversal.
Traffic stays inside AWS network infrastructure.

In simple terms:
Two isolated networks become directly connected.

Example:

- VPC-A: 10.0.0.0/16
- VPC-B: 192.168.0.0/16

After peering, an EC2 instance in VPC-A can directly reach an EC2 in VPC-B.

2. Why VPC Peering Exists

AWS isolates VPC networks by default. Without peering, VPCs cannot communicate.

Peering is used when you want:

- Communication between multiple environments
- Communication between different AWS accounts
- Communication between different regions

Common cases:

- Dev VPC ↔ Production VPC
- Application VPC ↔ Database VPC

- Shared services VPC ↔ Multiple project VPCs

3. Types of VPC Peering

Intra-Region VPC Peering

Both VPCs are in the same AWS region.

Example
Mumbai region:

```
VPC-A (10.0.0.0/16)
        |
        | Peering
        |
VPC-B (172.16.0.0/16) Characteristics:
```

- Low latency
- No cross-region cost
- Simple routing

Inter-Region VPC Peering VPCs

are in different AWS regions.

Example:

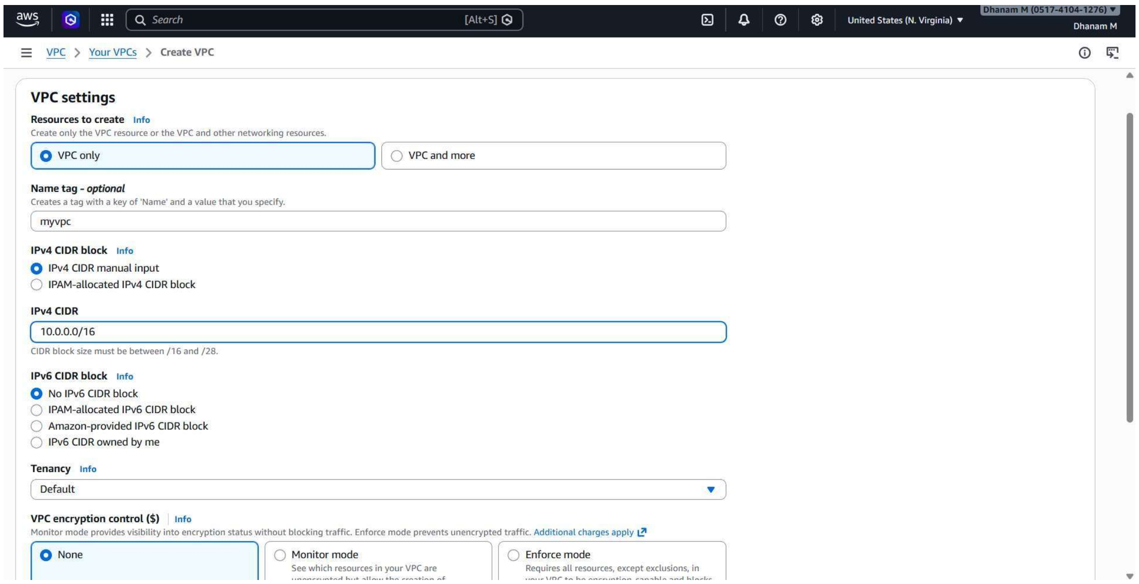
```
VPC-A (Mumbai)
        |
        | Peering
        |
VPC-B (Singapore) AWS encrypts the
```

traffic automatically.

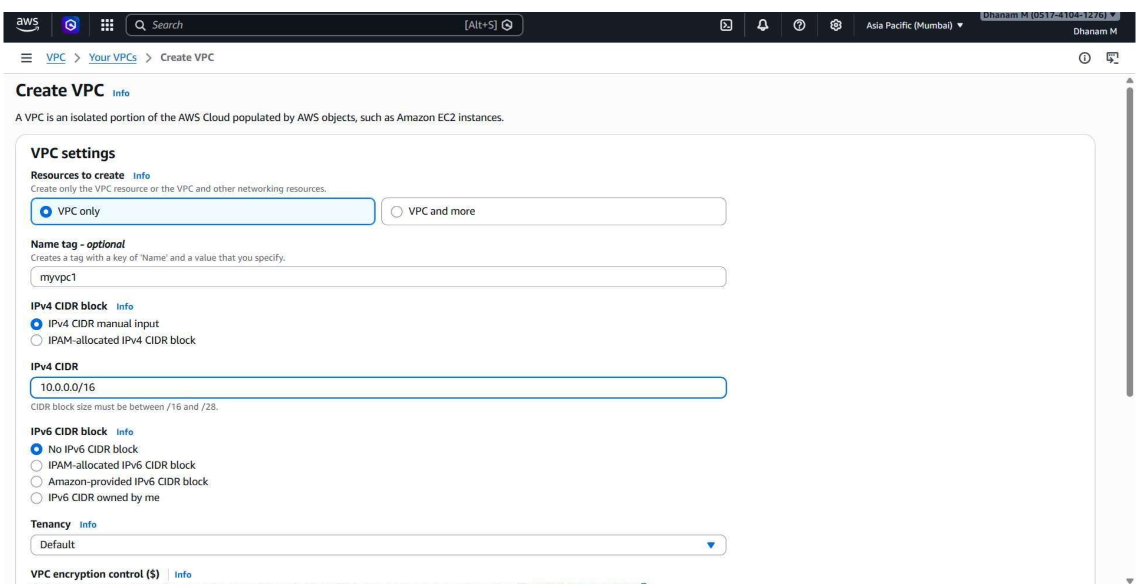
Used for:

- Multi-region architectures
- Disaster recovery setups

I have created one VPC in the N.virginia region.



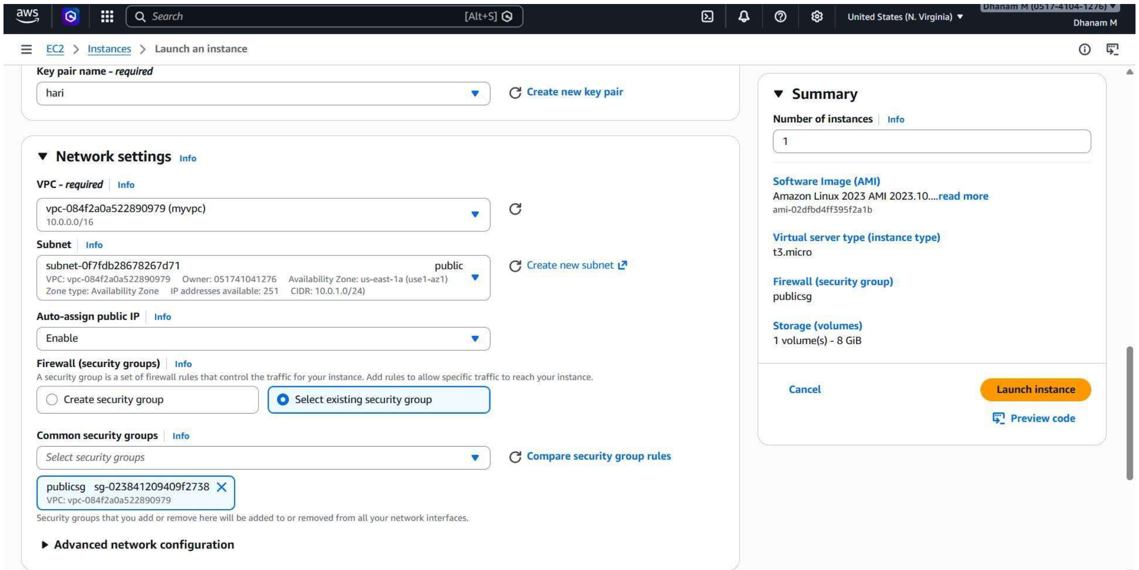
The other in the Mumbai region.



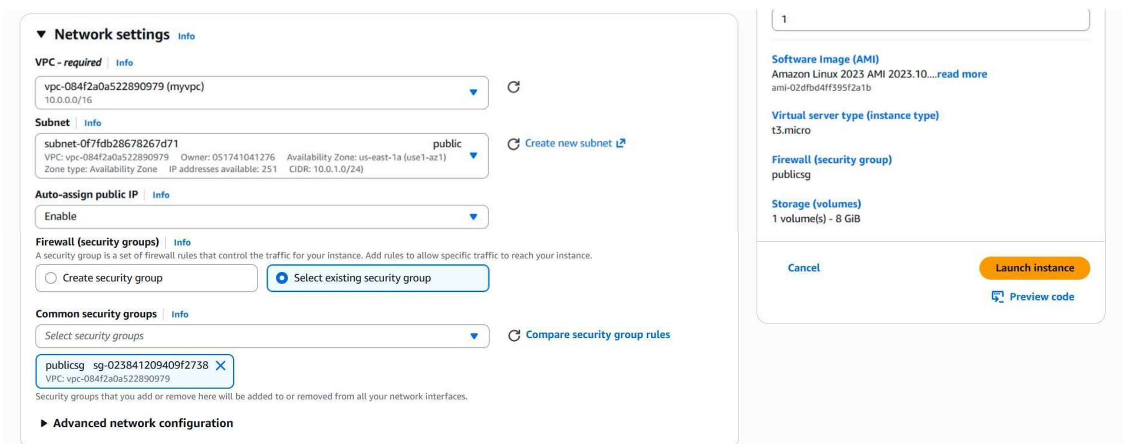
And I have configured all the recommended settings for the VPC to Activate.

(i.e) internet gateway , security groups , subnets.

The creation of public instance in the N.virginia region.

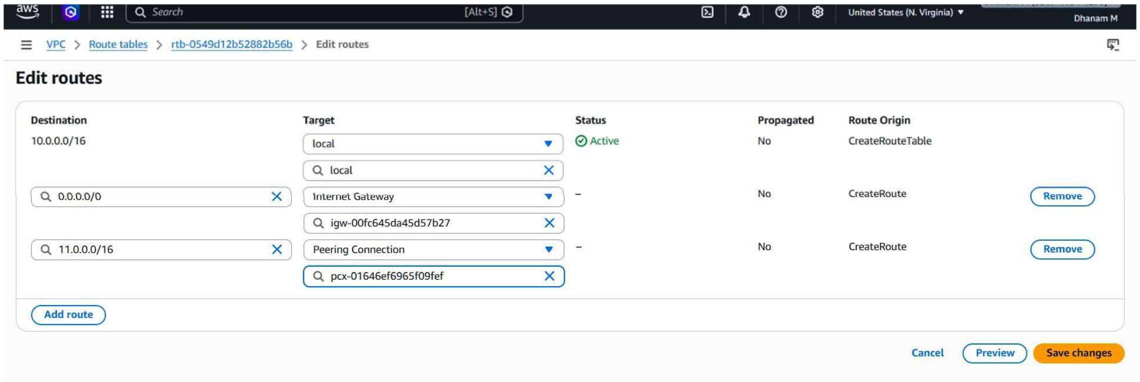


The creation of public instance in the Mumbai region.

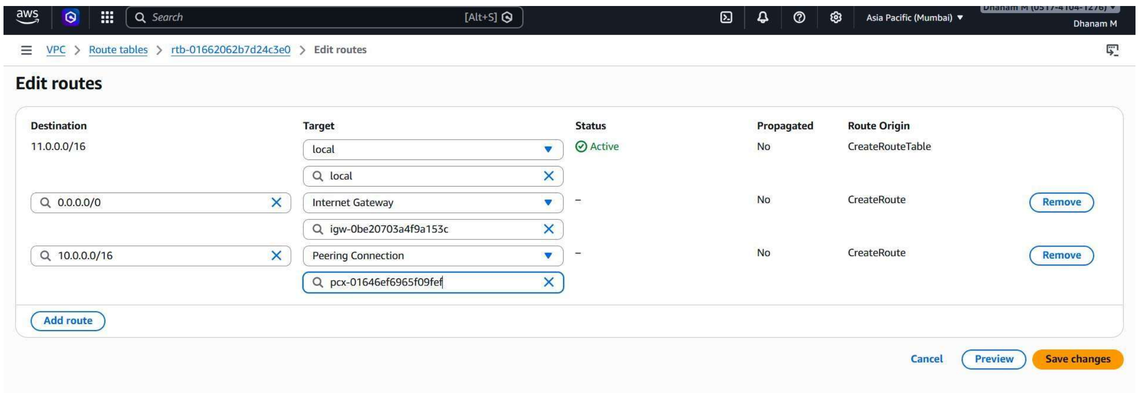


Before peering we have to give one more route values to all the VPC’s.

1st VPC.

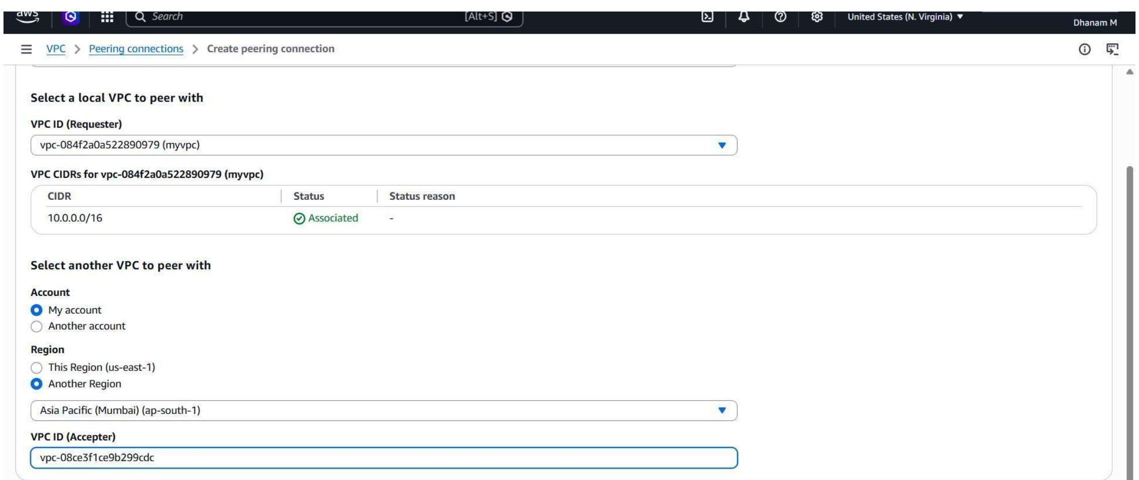


2nd VPC.

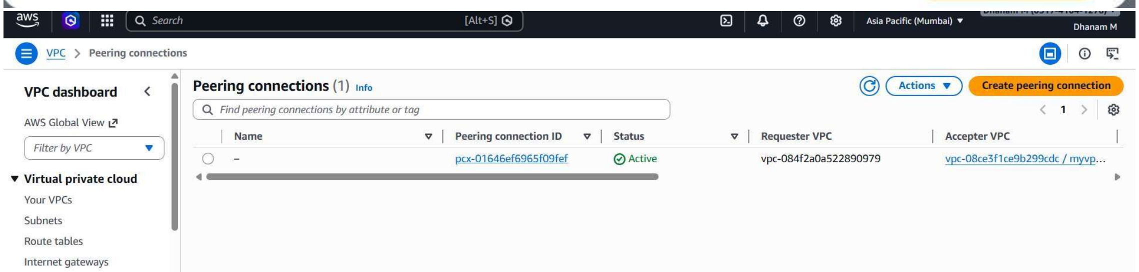


Then this is the Peering of the two VPC's From Different Regions.

This is the requesting form from the N.virginia Region.

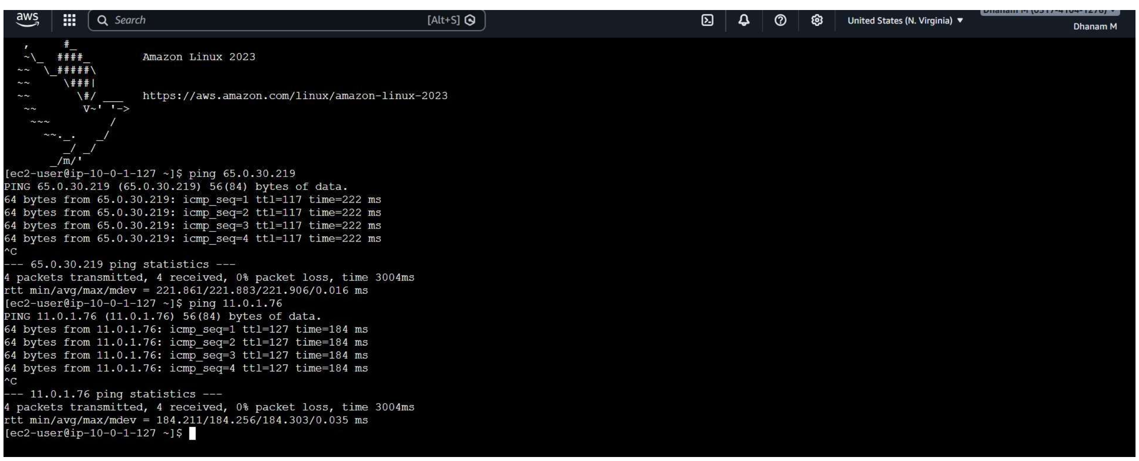


This is the Acceptor form in the Mumbai Region.



We are using ping command to check whether the internet is connected with the two VPC.

1st Instance.



2nd Instance.

Search [Alt+S] Asia Pacific (Mumbai) Dhanam M

```
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-11-0-1-76 ~]$ ping 13.223.100.210
PING 13.223.100.210 (13.223.100.210) 56(84) bytes of data.
64 bytes from 13.223.100.210: icmp_seq=1 ttl=108 time=222 ms
64 bytes from 13.223.100.210: icmp_seq=2 ttl=108 time=222 ms
64 bytes from 13.223.100.210: icmp_seq=3 ttl=108 time=222 ms
64 bytes from 13.223.100.210: icmp_seq=4 ttl=108 time=222 ms
^C
--- 13.223.100.210 ping statistics ---
5 packets transmitted, 4 received, 20% packet loss, time 4005ms
rtt min/avg/max/mdev = 221.879/221.905/221.919/0.015 ms
[ec2-user@ip-11-0-1-76 ~]$ ping 10.0.1.127
PING 10.0.1.127 (10.0.1.127) 56(84) bytes of data.
64 bytes from 10.0.1.127: icmp_seq=1 ttl=127 time=184 ms
64 bytes from 10.0.1.127: icmp_seq=2 ttl=127 time=184 ms
64 bytes from 10.0.1.127: icmp_seq=3 ttl=127 time=184 ms
64 bytes from 10.0.1.127: icmp_seq=4 ttl=127 time=184 ms
^C
--- 10.0.1.127 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 184.231/184.259/184.317/0.034 ms
[ec2-user@ip-11-0-1-76 ~]$
```